

The methods `getinit` and `getnext` are designed to be used in code like this, where `myhash` is a variable of type `Mhash`:

Retrieve all elements `e1` stored under a single key and do something with them.

```
if (myhash.getinit(&key)) {
    while (myhash.getnext(e1)) {
        Here use the returned element e1.
    }
}
```

7.6.5 Usage Examples

Having exposed in such detail the inner workings of the `Hash` and `Mhash` classes, we may have left the impression that these are difficult to use. Quite the contrary. Here's a piece of code that declares a hash memory for integers, and then stores the birth years of some personages:

```
Hash<string,Int,Hashfn2> year(1000,1000);

year[string("Marie Antoinette")] = 1755;
year[string("Ludwig van Beethoven")] = 1770;
year[string("Charles Babbage")] = 1791;
```

As declared, `year` can hold up to 1000 entries. We use the C++ `string` class as the key type. If we want to know how old Marie was when Charles was born, we can write,

```
Int diff = year[string("Charles Babbage")] - year[string("Marie Antoinette")];
cout << diff << '\n';
```

which prints "36".

Instead of using the C++ `string` class, you can, if you must, use null terminated C strings as keys, like this:

```
Hash<char,Int,Hashfn2> yearc(1000,1000);
yearc["Charles Babbage"][0] = 1791;
```

This works because `Hashfn2` has a special tweak, mentioned above, for key types that are apparently one byte long. Note the required use of `[0]` to send only the first byte of the C string; but that byte is passed by address, so `Hashfn2` knows where to find the rest of the string. (The syntax `yearc["Charles Babbage"]` is equivalent, also sending the first byte.)

Suppose we want to go the other direction, namely store the names of people into a hash memory indexed by birth year. Since more than one person may be born in a single year, we want to use a hash multimap memory, `Mhash`:

```
Mhash<Int,string,Hashfn2> person(1000,1000);

person.store(1775, string("Jane Austen"));
person.store(1791, string("Charles Babbage"));
person.store(1767, string("Andrew Jackson"));
person.store(1791, string("James Buchanan"));
person.store(1767, string("John Quincy Adams"));
person.store(1770, string("Ludwig van Beethoven"));
person.store(1791, string("Samuel Morse"));
person.store(1755, string("Marie Antoinette"));
```

It doesn't matter, of course, the order in which we put the names into the hash. Here is a piece of code to loop over years, printing the people born in that year:

```
string str;
for (Int i=1750;i<1800;i++) {
    if (person.getinit(i)) {
        cout << '\n' << "born in " << i << ":\n";
        while (person.getnext(str)) cout << str.data() << '\n';
    }
}
```

which gives as output

```
born in 1755:
Marie Antoinette

born in 1767:
Andrew Jackson
John Quincy Adams

born in 1770:
Ludwig van Beethoven

born in 1775:
Jane Austen

born in 1791:
Charles Babbage
James Buchanan
Samuel Morse
```

Notice that we could *not* have used null terminated C strings in this example, because C++ does not regard them as *first-class objects* that can be stored as elements of a vector. When you are using Hash or Mhash with strings, you will usually be better off using the C++ string class.

In §21.2 and §21.8 we will make extensive use of both the Hash and Mhash classes and almost all their member functions; look there for further usage examples.

By the way, Mr. Hacher's name is from the French *hacher*, meaning “to mince or hash.”

CITED REFERENCES AND FURTHER READING:

- Stroustrup, B. 1997, *The C++ Programming Language*, 3rd ed. (Reading, MA: Addison-Wesley), §17.6.2.[1]
- Knuth, D.E. 1997, *Sorting and Searching*, 3rd ed., vol. 3 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley), §6.4–§6.5.
- Vitter, J.S., and Chen, W-C. 1987, *Design and Analysis of Coalesced Hashing* (New York: Oxford University Press).

7.7 Simple Monte Carlo Integration

Inspirations for numerical methods can spring from unlikely sources. “Splines” first were flexible strips of wood used by draftsmen. “Simulated annealing” (we shall

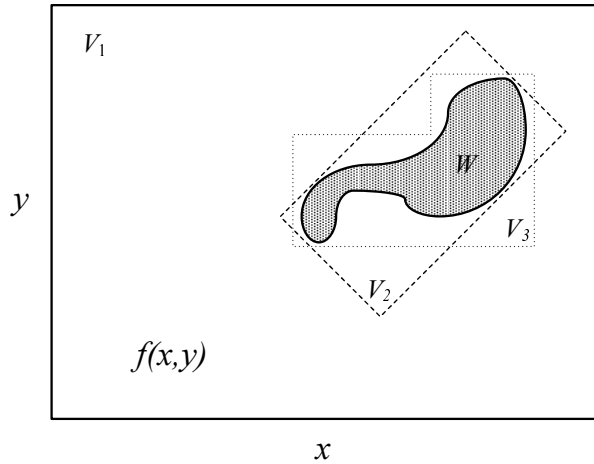


Figure 7.7.1. Monte Carlo integration of a function $f(x, y)$ in a region W . Random points are chosen within an area V that includes W and that can easily be sampled uniformly. Of the three possible V 's shown, V_1 is a poor choice because W occupies only a small fraction of its area, while V_2 and V_3 are better choices.

see in §10.12) is rooted in a thermodynamic analogy. And who does not feel at least a faint echo of glamor in the name “Monte Carlo method”?

Suppose that we pick N random points, uniformly distributed in a multidimensional volume V . Call them $x_0, \dots, x_{N-1}$. Then the basic theorem of Monte Carlo integration estimates the integral of a function f over the multidimensional volume,

$$\int f \, dV \approx V \langle f \rangle \pm V \sqrt{\frac{\langle f^2 \rangle - \langle f \rangle^2}{N}} \quad (7.7.1)$$

Here the angle brackets denote taking the arithmetic mean over the N sample points,

$$\langle f \rangle \equiv \frac{1}{N} \sum_{i=0}^{N-1} f(x_i) \quad \langle f^2 \rangle \equiv \frac{1}{N} \sum_{i=0}^{N-1} f^2(x_i) \quad (7.7.2)$$

The “plus-or-minus” term in (7.7.1) is a one standard deviation error estimate for the integral, not a rigorous bound; further, there is no guarantee that the error is distributed as a Gaussian, so the error term should be taken only as a rough indication of probable error.

Suppose that you want to integrate a function g over a region W that is not easy to sample randomly. For example, W might have a very complicated shape. No problem. Just find a region V that *includes* W and that *can* easily be sampled, and then define f to be equal to g for points in W and equal to zero for points outside of W (but still inside the sampled V). You want to try to make V enclose W as closely as possible, because the zero values of f will increase the error estimate term of (7.7.1). And well they should: Points chosen outside of W have no information content, so the effective value of N , the number of points, is reduced. The error estimate in (7.7.1) takes this into account.

Figure 7.7.1 shows three possible regions V that might be used to sample a complicated region W . The first, V_1 , is obviously a poor choice. A good choice, V_2 ,